ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных технологий
Кафедра Информатики и информационных технологий

направление подготовки
09.04.02 «Информационные системы и технологии»

Лабораторная работа №1

Дисциплина: Искусственный интеллект в мобильных системах.

Тема: Языки программирования для систем искуственного интеллекта в мобильных приложениях.

Выполнил: студент группы 244-331
Гусев Леонид Васильевич
(Фамилия И.О.)

Дата, подпись: 08.04.2026 _____
(Дата) (Подпись)

Проверил: _____ _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись: _____ _____
(Дата) (Подпись)

Замечания:

_____

_____

_____

Москва

2026

ОГЛАВЛЕНИЕ

Задание 3
Задача 1. Игра «Угадай число».....4
Задача 2. Игра «Угадай слово» 5
Задача 3. Генерация случайного пароля 6
Задача 4. Игра «Камень, ножницы, бумага» 7
Задача 5. Последовательность Фибоначчи.....8
Задача 6. Простые числа 9
Задача 7. Калькулятор 10
Задача 8. Игра «21 point» 11
Задача 9. Крестики-нолики.....12
Задача 10. Квадратное уравнение 13
Задача 11. Тип треугольника.....14
Задача 12. Табулирование функции 15
Вывод.....16

Задание

1) Изучите внимательно лекциию и методическое пособие по языку Python.

2) Выполните следующие задания и включите их решение в отчет. Отчет содержит титульный лист, и для каждой задачи: формулировка задания, код программы, скриншоты выполнения, пояснения к ним и выводы.

3) Прикрепите(загрузите) файл отчета для проверки.

Программный код доступен по ссылке - https://github.com/nialegu/ai_ms_labs/tree/main/1

Задача 1. Игра «Угадай число»

Описание программы: компьютер загадывает число от 1 до 100, после каждой попытки игрока компьютер будет говорить, больше ли загаданное число или меньше, пока не угадает.

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и измените программу так, чтобы играли двое – компьютер и человек по очереди, делая ходы, и кто первый угадал – тот и выиграл.

После доработки кода, с помощью функции input_number() происходит валидация пользовательского ввода. Добавлена логика игры между пользователем и компьютером - игра заканчивается, когда кто-то из двоих первым угадывает загаданное другим число.

```
Компьютер загадал число от 1 до 100!
Вы загадываете число: 50
Ваш ход: 25
Больше!
Ход компьютера: 5
Для компьютера: больше
Ваш ход: 75
Меньше!
Ход компьютера: 77
Для компьютера: меньше
Ваш ход: 50
Меньше!
Ход компьютера: 25
Для компьютера: больше
Ваш ход: 37
Меньше!
Ход компьютера: 40
Для компьютера: больше
Ваш ход: 30
Больше!
Ход компьютера: 65
Для компьютера: меньше
Ваш ход: 34
Вы угадали! Победа игрока!
```

Результат работы программы

Задача 2. Игра «Угадай слово»

Описание программы: угаданные буквы в слове будут открываться, но неудачные попытки буду отнимать жизни.

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и измените программу так, чтобы увеличить число слов, а также, чтобы человек играл с компьютером, угадывая слова из списка по очереди. Кто первый угадал, тот и выиграл.

Реализована игра в виселицу, расширенная до режима «человек против компьютера». Каждый из участников угадывает своё слово, выбранное случайным образом из расширенного списка (15 слов). Компьютер перебирает буквы алфавита в случайном порядке. У каждого игрока по 5 жизней. Тот, кто первым угадает своё слово, выигрывает.

```
Игра: человек против компьютера!
Выберите слово из списка ['яблоко', 'победа'

Компьютер загадал своё слово.

Ваше слово для угадывания: _ _ _ _ _ _
Введите букву: к
Буквы нет. Жизни: 4

Ход компьютера...
Компьютер вводит: щ
Буквы нет. Жизни компьютера: 4

Ваше слово для угадывания: _ _ _ _ _ _
Введите букву: а
Буква есть!

Ход компьютера...
Компьютер вводит: ё
Буквы нет. Жизни компьютера: 3

Ваше слово для угадывания: _ _ _ _ _ а
Введите букву: и
Буквы нет. Жизни: 3
```

```
Ход компьютера...
Компьютер вводит: э
Буквы нет. Жизни компьютера: 2

Ваше слово для угадывания: _ _ _ _ _ а
Введите букву: т
Буквы нет. Жизни: 2

Ход компьютера...
Компьютер вводит: б
Буквы нет. Жизни компьютера: 1

Ваше слово для угадывания: _ _ _ _ _ а
Введите букву: ф
Буквы нет. Жизни: 1

Ход компьютера...
Компьютер вводит: о
Буквы нет. Жизни компьютера: 0

У компьютера закончились жизни. Слово было "система".
```

Результат работы программы

Задача 3. Генерация случайного пароля

Описание программы: Программа создает пароль из набора случайных символов.

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и измените программу так, чтобы была функция генерации пароля, а пользователь в цикле (пока не введен символ “q”) вводил сайт и логин, а программа генерировала пароль и сохраняла (добавляла) в файл построчно тройку значений сайт-логин-пароль.

Программа содержит функцию generate_password(), которая создает случайный пароль из заглавных и строчных букв, цифр и специальных символов. В цикле пользователь вводит название сайта и логин, после чего генерируется пароль и добавляется в файл passwords.txt в формате сайт - логин - пароль. Ввод символа «q» завершает работу программы.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_3.py
Введите сайт (или q для выхода): google.com
Введите логин: login
Сгенерированный пароль: B7{s_&FH60?Brw
Введите сайт (или q для выхода): q
Выход из программы.
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_3.py
Введите сайт (или q для выхода): vk.ru
Введите логин: login
Сгенерированный пароль: x]Y3eJ"pOH5_w
Введите сайт (или q для выхода): t.me
Введите логин: login
Сгенерированный пароль: g?S86*j>Vz6Qs(
```

```
.gitignore    task_3.py U    passwords.txt U
1 > passwords.txt
  1   google.com - login - B7{s_&FH60?Brw
  2   vk.ru - login - x]Y3eJ"pOH5_w
  3   t.me - login - g?S86*j>Vz6Qs(
  4
```

Задача 4. Игра «Камень, ножницы, бумага»

Описание программы: пользователь загадывает одно из трех камень, или ножницы или бумагу. Вводит свое решение в консоль, а затем компьютер случайным образом определяет свой выбор. В дальнейшем, с помощью условий определяется, кто победил, результат игры выводится на экран.

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и добавьте «колодец». Правила просты – бумага побеждает колодец (накрывает), а колодец побеждает камень и ножницы (они тонут).

Программа реализует игру «Камень, Ножницы, Бумага» с добавлением «Колодца». Пользователь вводит свой ход, компьютер выбирает случайный вариант. Затем программа определяет победителя по правилам: Бумага накрывает Колодец, Колодец тонет Камень и Ножницы, остальные правила стандартные. Результат выводится на экран. Ввод «q» завершает игру.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_4.py
Игра Камень, Ножницы, Бумага, Колодец!
Правила: Бумага побеждает Колодец, Колодец побеждает Камень и Ножницы.
Выберите Камень, Бумага, Ножницы или Колодец (q для выхода): dfgdd
Некорректный ход! Попробуйте снова.
Выберите Камень, Бумага, Ножницы или Колодец (q для выхода): Камень
Компьютер выбрал: Ножницы
Ты выиграл! Камень разбивает Ножницы
Выберите Камень, Бумага, Ножницы или Колодец (q для выхода): Колодец
Компьютер выбрал: Камень
Ты выиграл! Колодец тонет Камень
Выберите Камень, Бумага, Ножницы или Колодец (q для выхода): Ножницы
Компьютер выбрал: Колодец
Ты проиграл! Ножницы тонут в Колодце
Выберите Камень, Бумага, Ножницы или Колодец (q для выхода): Бумага
Компьютер выбрал: Бумага
Ничья!
Выберите Камень, Бумага, Ножницы или Колодец (q для выхода): q
Выход из игры.
```

Задача 5. Последовательность Фибоначчи

Описание программы: Числа Фибоначчи – это ряд чисел, в котором каждое следующее число равно сумме двух предыдущих: 1, 1, 2, 3, 5, 8, 13, ... Иногда ряд начинают с нуля: 0, 1, 1, 2, 3, 5, ... В данном случае мы будем придерживаться первого варианта. Пользователю необходимо ввести произвольное число, далее, программа выводит ряд чисел.

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и потом напишите новую функцию fibRecurse, которая использует рекурсивный алгоритм вычисления чисел Фибоначчи, который основан на рекуррентных отношениях чисел Фибоначчи:

$$F_1 = F_2 = 1,\ F_n = F_{n-1} + F_{n-2}.$$

Программа генерирует ряд чисел Фибоначчи. Пользователь вводит количество чисел, после чего программа выводит ряд двумя способами:

1. Итеративно с помощью функции fibSequence
2. Рекурсивно с помощью функции fibRecurse, которая вычисляет каждое число по формуле:

$F_n = F_{n-1} + F_{n-2}$, где $F_1 = F_2 = 1$

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_5.py
Сколько чисел Фибоначчи вывести? 10
Ряд Фибоначчи (итеративно): 1, 1, 2, 3, 5, 8, 13, 21, 34, 55
Ряд Фибоначчи (рекурсивно): 1, 1, 2, 3, 5, 8, 13, 21, 34, 55
```

Задача 6. Простые числа

Описание программы: Простое число — это число, у которого только два делителя: 1 и само число. Программа запрашивает у пользователя разрешение на вывод следующего просто числа.

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и измените программу так, чтобы она запрашивала у пользователя число и отвечала – простое оно или нет, и, если число не простое, то выводила на экран БЛИЖАЙШЕЕ простое число.

Программа проверяет, является ли введённое пользователем число простым. Если число простое, выводится соответствующее сообщение. Если число не простое, программа находит и выводит ближайшее простое число.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_6.py
Введите число для проверки: 10
10 — не простое число.
Ближайшее простое число: 11
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_6.py
Введите число для проверки: 11
11 — простое число!
```

Задача 7. Калькулятор

Описание программы: программа выполняет над двумя вещественными числами одну из четырех арифметических операций (сложение, вычитание, умножение или деление).

Задание: Отладьте код программы, зафиксируйте работу программы скриншотами в отчете и добавьте в калькулятор операции: возведение в степень, вычисление процента числа, извлечение корня.

Программа реализует калькулятор, который выполняет операции: сложение, вычитание, умножение, деление, возведение в степень, вычисление процента числа и извлечение квадратного корня. Пользователь вводит числа и выбирает операцию, после чего программа выводит результат. Для операции корня второе число не требуется.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_7.py
Калькулятор: +, -, *, /, ^ (возведение в степень), % (процент), √ (корень)
Введите первое число: 5
Выберите операцию: +, -, *, /, ^, %, √ : %
Введите второе число: 100
100.0% от 5.0 = 5.0
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_7.py
Калькулятор: +, -, *, /, ^ (возведение в степень), % (процент), √ (корень)
Введите первое число: 4
Выберите операцию: +, -, *, /, ^, %, √ : √
√4.0 = 2.0
```

Задача 8. Игра «21 point»

Описание программы: Изначально у пользователя 0 очков. Программа спрашивает у пользователя, хочет ли он взять карту. Пользователь должен ввести в консоль либо y – yes, либо n – no. Если пользователь ответил n, то программа озвучивает набранное количество очков и завершает свою работу. Если пользователь ввел команду “y”, то игра продолжается и выдается 1 карта из списка. Далее, программа прибавляет к числу очков снятую карту и выводит количество очков. Если количество очков больше 21, то пользователь проигрывает, и программа завершает свою работу. Если число очков равно 21, то пользователь выиграл. Если меньше - программа снова предлагает взять карту. После окончания игры, программа прощается с пользователем.

Программа реализует игру «21» между игроком и ботом. Игрок берёт карты до 21 очка. Бот ходит после игрока и выбирает карту с учётом коэффициента сложности: чем выше коэффициент (от 0 до 1), тем чаще бот подбирает карту, чтобы выиграть. Программа выводит результат игры: победа, поражение или ничья, после чего завершается.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_8.py
Введите уровень сложности бота (0-1): 1
Поиграем в 21?
Если хотите играть нажмите Enter, если хотите выйти, то Ctrl+C

Будете ли вы брать карту? (yes/no): yes
Вы взяли карту: 6
Сейчас у вас 6 очков
Будете ли вы брать карту? (yes/no):
```

```
Вы взяли карту: 7
Сейчас у вас 20 очков
Будете ли вы брать карту? (yes/no): no
У вас 20 очков.
Ход бота...
Бот берет карту: 6
У бота 6 очков
Бот берет карту: 4
У бота 10 очков
Бот берет карту: 11
У бота 21 очков
Бот победил! У него 21 очков, у вас 20
Нажмите Enter, чтобы закрыть игру.
```

Задача 9. Крестики-нолики

Программа реализует игру «Крестики-нолики» в двух режимах:

1. Два игрока на поле 5×5 – игроки поочередно делают ходы, и программа проверяет победу по строкам, столбцам и диагоналям.
2. Игрок против компьютера на поле 3×3 – игрок ходит «X», компьютер случайно ставит «O». Игра проверяет победителя или ничью после каждого хода.

Ввод осуществляется через номера свободных клеток, а поле обновляется в терминале после каждого хода.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_9.py
Выберите режим игры:
1 - Два игрока (5x5)
2 - Игрок против компьютера (3x3)
Ваш выбор: 2
-------------
| 1 | 2 | 3 |
-------------
| 4 | 5 | 6 |
-------------
| 7 | 8 | 9 |
-------------
Куда поставим X? 2
-------------
| 1 | X | 3 |
-------------
| 4 | 5 | 6 |
-------------
| 7 | 8 | 9 |
-------------
Компьютер поставил O в клетку 1
-------------
| O | X | 3 |
-------------
| 4 | 5 | 6 |
-------------
| 7 | 8 | 9 |
-------------
```

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_9.py
| O | X | O |
-------------
| X | 5 | 6 |
-------------
| 7 | 8 | 9 |
-------------
Куда поставим X?
Некорректный ввод. Введите число.
Куда поставим X? 5
-------------
| O | X | O |
-------------
| X | X | 6 |
-------------
| 7 | 8 | 9 |
-------------
Компьютер поставил O в клетку 9
-------------
| O | X | O |
-------------
| X | X | 6 |
-------------
| 7 | 8 | O |
-------------
Куда поставим X? 8
-------------
| O | X | O |
-------------
| X | X | 6 |
-------------
| 7 | X | O |
-------------
X выиграл!
```

Задача 10. Квадратное уравнение

Программа решает квадратное уравнение $ax^2 + bx + c = 0$ с учётом всех вариантов коэффициентов:

• Если $a = 0$, проверяется, является ли уравнение линейным или не имеет решений.

• Вычисляется дискриминант $D = b^2 - 4ac$.

• В зависимости от значения D выводятся:

• Два действительных корня ($D > 0$)

• Один действительный корень ($D = 0$)

• Два комплексных корня ($D < 0$)

Программа корректно обрабатывает все случаи, включая нулевые коэффициенты.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_10.py
Введите коэффициент a: 5
Введите коэффициент b: 10
Введите коэффициент c: 25

Рассматриваем уравнение: 5.0x² + 10.0x + 25.0 = 0

Два комплексных корня: x1 = -1.0+2.0i, x2 = -1.0-2.0i
```

Задача 11. Тип треугольника

Программа определяет тип треугольника по сторонам и углам:

1. Сначала проверяет, существует ли треугольник (сумма любых двух сторон больше третьей).
2. Определяет тип углов:
    - прямоугольный (один угол 90°)
    - остроугольный (все углы < 90°)
    - тупоугольный (один угол > 90°)
3. Определяет тип сторон:
    - равносторонний (все стороны равны)
    - равнобедренный (две стороны равны)
    - разносторонний (все стороны разные)
4. Выводит комбинированный результат, например:
    - Треугольник равнобедренный-прямоугольный
    - Треугольник разносторонний-остроугольный

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_11.py
Введите сторону a: 3
Введите сторону b: 4
Введите сторону c: 5
Треугольник разносторонний-прямоугольный.
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_11.py
Введите сторону a: 5
Введите сторону b: 5
Введите сторону c: 5
Треугольник равносторонний-остроугольный.
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_11.py
Введите сторону a: 1
Введите сторону b: 2
Введите сторону c: 10
Треугольник с такими сторонами не существует.
```

Задача 12. Табулирование функции

Программа табулирует функцию $y = x^2 - 2x + 1$ на заданном пользователем интервале [A, B] с шагом H.

• Используется форматированный вывод для наглядности, с выравниванием столбцов и 4 знаками после запятой.

• Программа проверяет корректность ввода и положительность шага.

• Результат выводится в виде таблицы x | y.

```
leonidgusev@MacBook-Pro-Leonid 1 % python3 task_12.py
Введите начало интервала A: -5
Введите конец интервала B: 10
Введите шаг H: 0.5

         x | y = x^2 - 2x + 1
----------------------------
   -5.0000 |         36.0000
   -4.5000 |         30.2500
   -4.0000 |         25.0000
   -3.5000 |         20.2500
   -3.0000 |         16.0000
   -2.5000 |         12.2500
   -2.0000 |          9.0000
   -1.5000 |          6.2500
   -1.0000 |          4.0000
   -0.5000 |          2.2500
    0.0000 |          1.0000
    0.5000 |          0.2500
    1.0000 |          0.0000
    1.5000 |          0.2500
    2.0000 |          1.0000
    2.5000 |          2.2500
    3.0000 |          4.0000
    3.5000 |          6.2500
    4.0000 |          9.0000
    4.5000 |         12.2500
    5.0000 |         16.0000
    5.5000 |         20.2500
    6.0000 |         25.0000
    6.5000 |         30.2500
    7.0000 |         36.0000
    7.5000 |         42.2500
    8.0000 |         49.0000
    8.5000 |         56.2500
    9.0000 |         64.0000
    9.5000 |         72.2500
   10.0000 |         81.0000
```

Вывод

В рамках выполнения практической работы было создано 12 программ на языке Python, охватывающих разнообразные задачи: игровые алгоритмы, математические вычисления, работу с файлами и форматированный вывод данных. Все разработанные программы прошли отладку и были расширены в соответствии с требованиями задания.

В процессе выполнения работы были закреплены навыки применения функций, рекурсии, циклов, обработки строк и списков, а также организации взаимодействия с пользователем в консольных приложениях. Приобретённый опыт формирует прочную основу для дальнейшей разработки более сложных алгоритмических модулей в системах искусственного интеллекта.